

Останин Андрей

Домашнее задание 1: SETCOVER

Первое, что проверим, - жадный алгоритм. На каждом шаге выбираем то множество, в котором стоимость еще не покрытого элемента минимальна (то есть вся стоимость множества распределяется равномерно на еще не покрытые элементы, берется множество с минимальной стоимостью одного элемента). Сразу добавим немного рандома: будем выбирать из 7 таких множеств (с минимальной удельной стоимостью) с некоторым распределением вероятности (попробую $\frac{1}{x^3}$ (нормированное), чтобы вероятность у лучших была выше). Так мы, разумеется, очень вряд ли достигнем оптимального покрытия, но зато, возможно, оставим какое-то пространство для маневра в дальнейшем, когда на некотором шаге возьмем не оптимальное. Трюк с вероятностью вводится из интереса и ради новизны (обычный жадник был на лекции, уже показывалось, что точность - $0.7 \log n$).

Таблица 1: Результаты тестирования SETCOVER. Стохастический жадник

Название теста	Count	Баллы
sc_157_0	106 600	3/5
sc_330_0	32	0/5
sc_1000_11	190	3/5
sc_5000_1	38	3/5
sc_10000_5	80	3/5
sc_10000_2	198	3/5
ИТОГО		15/30

Сравним с жадником:

Таблица 2: Результаты тестирования SETCOVER. Жадник

Название теста	Count	Баллы
sc_157_0	102 100	3/5
sc_330_0	30	0/5
sc_1000_11	170	3/5
sc_5000_1	36	3/5
sc_10000_5	76	3/5
sc_10000_2	192	3/5
ИТОГО		15/30

К сожалению, никакого преимущества над жадником не получили. Рассмотрим теперь это как задачу линейного программирования:

$$\min_x \langle x, c \rangle \text{ s.t. } \sum_{i=0}^{n-1} \mathbf{I}[j \in S_i] \cdot x_i \geq 1, x \in [0; 1]^{|S|}$$

Зафиксируем λ . На лекции мы имели дело с $c = \mathbf{1}$. Попробуем использовать решение с лекции, а также попробуем его модифицировать. Здесь попробуем для нецелых x_i брать S_i в покрытие с вероятностью $\min\{1, \lambda x_i\}$. Для решения воспользуемся **ortools**. Этим решением получилось улучшить результат только для **sc_157_0**. Для остальных тестов - по нулям. Так же я еще попробовал сначала использовать солвер для ЛП, затем добивать жадником: аналогично, улучшилось только на **sc_157_0** (там очень много зануляется), на остальных результаты как у жадника. Появилась идея: генерируя случайные векторы из 0 и 1 мы имеем случай, когда сгенерировался плохой вектор (не покрывает), и нам приходится генерировать новый; будем вместо этого генерировать перестановку (x_i сопоставляем $\mathcal{U}[0, x_i]$, затем сортируем случайные величины по убыванию), идти по ней, пока множество

не покрылось. Получили значимое улучшение по сравнению с другими подходами с ЛП, но все так же проходит только `sc_157_0`.

Таблица 3: Результаты тестирования SETCOVER. ЛП + много перестановок

Название теста	Count	Баллы
sc_157_0	95 300	3/5
sc_330_0	330	0/5
sc_1000_11	446	0/5
sc_5000_1	142	0/5
sc_10000_5	331	0/5
sc_10000_2	874	0/5
ИТОГО		3/30

Теперь попробуем использовать решение ЛП (нецелые коэффициенты) в качестве эвристики для beam search. Будем рассматривать множества в порядке убывания коэффициента, полученного в решении ЛП. Поддерживать будем 10 лучших путей (хорошесть оценивается как ответ жадника, основанном на этом пути: чем меньше ответ, тем лучше путь). Для начала я использую в качестве эвристики стоимость множеств (не удельную) - мне лень на паре качать плюсовые `ortools` (отмечу, что, скорее всего, придется ифать размер входа). Результаты первого запуска ниже.

Таблица 4: Результаты тестирования SETCOVER. Beam Search: 10 путей.

Название теста	Count	Баллы
sc_157_0	96 000	3/5
sc_330_0	25	3/5
sc_1000_11	155	3/5
sc_5000_1	TL	0/5
sc_10000_5	TL	0/5
sc_10000_2	TL	0/5
ИТОГО		9/30

Увеличение числа путей не дало сильного преимущества:

Таблица 5: Результаты тестирования SETCOVER. Beam Search: 30 путей.

Название теста	Count	Баллы
sc_157_0	96 000	3/5
sc_330_0	25	3/5
sc_1000_11	152	3/5
sc_5000_1	TL	0/5
sc_10000_5	TL	0/5
sc_10000_2	TL	0/5
ИТОГО		9/30

Изменив эвристику на коэффициенты в ЛП улучшений не получили. Таким образом, на текущий момент мы имеем 18 баллов (если делать иф по размеру входа: маленький $\Rightarrow$ beam search, иначе - жадник).

Таблица 6: Результаты тестирования SETCOVER. Beam Search: 10 путей. Эвристика - ЛП

Название теста	Count	Баллы
sc_157_0	102 100	3/5
sc_330_0	30	0/5
sc_1000_11	170	3/5
sc_5000_1	TL	0/5
sc_10000_5	TL	0/5
sc_10000_2	TL	0/5
ИТОГО		6/30

Следующее улучшение: я думаю, что чем ближе множества к началу рассмотрения, тем

сильнее его влияние на дальнейшие ответы. Поэтому попробуем амплифицировать решения beam search: отправим все элемента из решения в конец рассмотрения! Так добавим новизны нашим решениям.

Таблица 7: Результаты тестирования SETCOVER. Beam Search: 5 путей. Амплификация: 12 повторений

Название теста	Count	Баллы
sc_157_0	95 800	3/5
sc_330_0	24	5/5
sc_1000_11	155	3/5
sc_5000_1	TL	0/5
sc_10000_5	TL	0/5
sc_10000_2	TL	0/5
ИТОГО		11/30

Добились оценки в пять баллов на втором тесте - идея принесла свои плоды. Сейчас в совокупности имеем:

Таблица 8: Результаты тестирования SETCOVER. Beam Search + Жадник

Название теста	Count	Баллы
sc_157_0	95 800	3/5
sc_330_0	24	5/5
sc_1000_11	155	3/5
sc_5000_1	37	3/5
sc_10000_5	79	3/5
sc_10000_2	196	3/5
ИТОГО		20/30

Следующая эвристика, которую хочется проверить: минимальный номер элемента в множестве. Идя по множествам в порядке увеличения этой статистики, мы оставляем множества, которые уже не будут трогать префикс (можно еще чуть-чуть изменить такую эвристику: отсортировать в порядке уменьшения числа вхождений элемента в множества, идти в порядке возрастания числа вхождений по элементам). Эти действия направлены на уменьшение степени свободы рассматриваемых решений.

Таблица 9: Результаты тестирования SETCOVER. Beam Search (less frequent + prefix) + Жадник

Название теста	Count	Баллы
sc_157_0	96 800	3/5
sc_330_0	24	5/5
sc_1000_11	156	3/5
sc_5000_1	TL	0/5
sc_10000_5	79	3/5
sc_10000_2	196	3/5
ИТОГО		17/30

(Была попытка все-таки запустить на sc_5000_1 beam search, но выполняется слишком долго (не завершилось за >20 минут))

Значимых улучшений не получено. Появилась идея динамической эвристики: на каждом этапе предпочитать множества, содержащие элемент с нахождением в наименьшем количестве оставшихся множеств (до этого мы рассматривали только статическую эвристику: все пути шли по одному и тому же дереву).

Решил написать генетику с tabu search (пока без crossover): Рейтинг множества - среднее по средним стоимостям множеств, в которых содержатся элементы множества (тут я тупанул на самом деле: надо было смотреть разность стоимости множества и среднего по средним стоимостям множеств (не включая текущего множества)). Мутация - баним слу-

чайные `max_banned_mutation` множеств в существующем решении. `tabu_tenure` - сколько поколений живет бан. `tour_k` - сколько особей мы сравним при формировании потомства (то есть выбирается из k случайных лучшая особь). `gen` - размер популяции.

Что происходит глобально: изначально популяция инициализируется случайно (каждое множество банится с вероятностью 15%). Далее мы выбираем особей, дающих потомство: очередная особь - эта лучшая особь по результатам турнира из `tour_k` туров. Эта особь мутирует (банятся какие-то из выбранных множеств), и баны складываются в `tabu` список. Через `tabu_tenure` поколений баны отменяются.

Таблица 10: Результаты тестирования SETCOVER. Tabu Search (`pop_size=24`, `tour_k=4`, `gen=450`, `max=5`, `tenure=9`) + Жадник (Рейтинги)

Название теста	Count	Баллы
sc_157_0	94,400	5/5
sc_330_0	23	5/5
sc_1000_11	168	3/5
sc_5000_1	TIMEOUT	0/5
sc_10000_5	TIMEOUT	0/5
sc_10000_2	TIMEOUT	0/5
ИТОГО		13/30

Таблица 11: Результаты тестирования SETCOVER. Tabu Search (`pop_size=24`, `tour_k=4`, `gen=450`, `max=7`, `tenure=6`) + Жадник (Рейтинги)

Название теста	Count	Баллы
sc_157_0	95,100	3/5
sc_330_0	27	3/5
sc_1000_11	160	3/5
sc_5000_1	TIMEOUT	0/5
sc_10000_5	TIMEOUT	0/5
sc_10000_2	TIMEOUT	0/5
ИТОГО		9/30

Таблица 12: Результаты тестирования SETCOVER. Tabu Search (`pop_size=16`, `tour_k=5`, `gen=450`, `max=3`, `tenure=8`) + Жадник (Рейтинги)

Название теста	Count	Баллы
sc_157_0	95,100	3/5
sc_330_0	23	5/5
sc_1000_11	149	3/5
sc_5000_1	TIMEOUT	0/5
sc_10000_5	TIMEOUT	0/5
sc_10000_2	TIMEOUT	0/5
ИТОГО		11/30

Таблица 13: Результаты тестирования SETCOVER. Tabu Search (`pop_size=16`, `tour_k=5`, `gen=550`, `max=3`, `tenure=8`) + Жадник (Рейтинги)

Название теста	Count	Баллы
sc_157_0	94,900	3/5
sc_330_0	23	5/5
sc_1000_11	149	3/5
sc_5000_1	TIMEOUT	0/5
sc_10000_5	TIMEOUT	0/5
sc_10000_2	TIMEOUT	0/5
ИТОГО		11/30

Таблица 14: Результаты тестирования SETCOVER. Tabu Search (pop_size=16, tour_k=5, gen=1000, max=3, tenure=8) + Жадник (Минимальная средняя стоимость элемента)

Название теста	Count	Баллы
sc_157_0	94,400	5/5
sc_330_0	23	5/5
sc_1000_11	147	5/5
sc_5000_1	30	5/5
sc_10000_5	TIMEOUT	0/5
sc_10000_2	TIMEOUT	0/5
ИТОГО		20/30

Таблица 15: Результаты тестирования SETCOVER. Tabu Search (pop_size=16, tour_k=5, gen=500, max=3, tenure=8) + Жадник (Минимальная средняя стоимость элемента)

Название теста	Count	Баллы
sc_157_0	94,400	5/5
sc_330_0	23	5/5
sc_1000_11	147	5/5
sc_5000_1	31	5/5
sc_10000_5	67	3/5
sc_10000_2	171	3/5
ИТОГО		26/30

(Стоит отметить, что решение нестабильно: небольшой сдвиг в гиперпараметрах приводит к уменьшению баллов)

Таблица 16: Результаты тестирования SETCOVER. Tabu Search (pop_size=16, tour_k=6, gen=500, max=3, tenure=8) + Жадник (Минимальная средняя стоимость элемента)

Название теста	Count	Баллы
sc_157_0	95,900	3/5
sc_330_0	27	3/5
sc_1000_11	147	5/5
sc_5000_1	31	5/5
sc_10000_5	67	3/5
sc_10000_2	171	3/5
ИТОГО		22/30

Ранее отмечал, что эвристика с рейтингами была не совсем корректна. Попробуем с обновленной функцией рейтингов.

Таблица 17: Результаты тестирования SETCOVER. Tabu Search (pop_size=16, tour_k=5, gen=500, max=3, tenure=8) + Жадник (Рейтинги v2)

Название теста	Count	Баллы
sc_157_0	94,600	3/5
sc_330_0	23	5/5
sc_1000_11	152	3/5
sc_5000_1	33	3/5
sc_10000_5	67	3/5
sc_10000_2	170	3/5
ИТОГО		20/30

Как мы видим, лучшей эвристикой остается выбор элемента с минимальной средней стоимостью элемента.

Таблица 18: Результаты тестирования SETCOVER. Tabu Search (pop_size=24, tour_k=5, gen=1000, max=3, tenure=2) + Жадник (Минимальная средняя стоимость элемента)

Название теста	Count	Баллы
sc_157_0	95,900	3/5
sc_330_0	23	5/5
sc_1000_11	148	3/5
sc_5000_1	31	5/5
sc_10000_5	64	5/5
sc_10000_2	170	3/5
ИТОГО		24/30

Таблица 19: Результаты тестирования SETCOVER. Tabu Search (pop_size=36, tour_k=5, gen=500, max=4, tenure=6) + Жадник (Минимальная средняя стоимость элемента)

Название теста	Count	Баллы
sc_157_0	95,900	3/5
sc_330_0	23	5/5
sc_1000_11	146	5/5
sc_5000_1	30	5/5
sc_10000_5	64	5/5
sc_10000_2	170	3/5
ИТОГО		24/30

Добавил crossover: дети наследуют табу обоих родителей (каждый родитель выбирается через k туров), ребенок формируется как пересечение родителей, мутация, а затем локальный поиск.

Таблица 20: Результаты тестирования SETCOVER. Tabu Search + Crossover (pop_size=36, tour_k=5, gen=500, max=4, tenure=6) + Жадник (Минимальная средняя стоимость элемента)

Название теста	Count	Баллы
sc_157_0	96,700	3/5
sc_330_0	23	5/5
sc_1000_11	146	5/5
sc_5000_1	30	5/5
sc_10000_5	64	5/5
sc_10000_2	167	5/5
ИТОГО		28/30

Была идея устраивать глобальные мутации (то есть запрещать всем на несколько поколений какие-то множества), но эта идея не принесла улучшений. Мы уже достигали второго порога на первом тесте, но хочется достичь его честным путем, без запуска второй генетики.